

Design Concepts and Principles



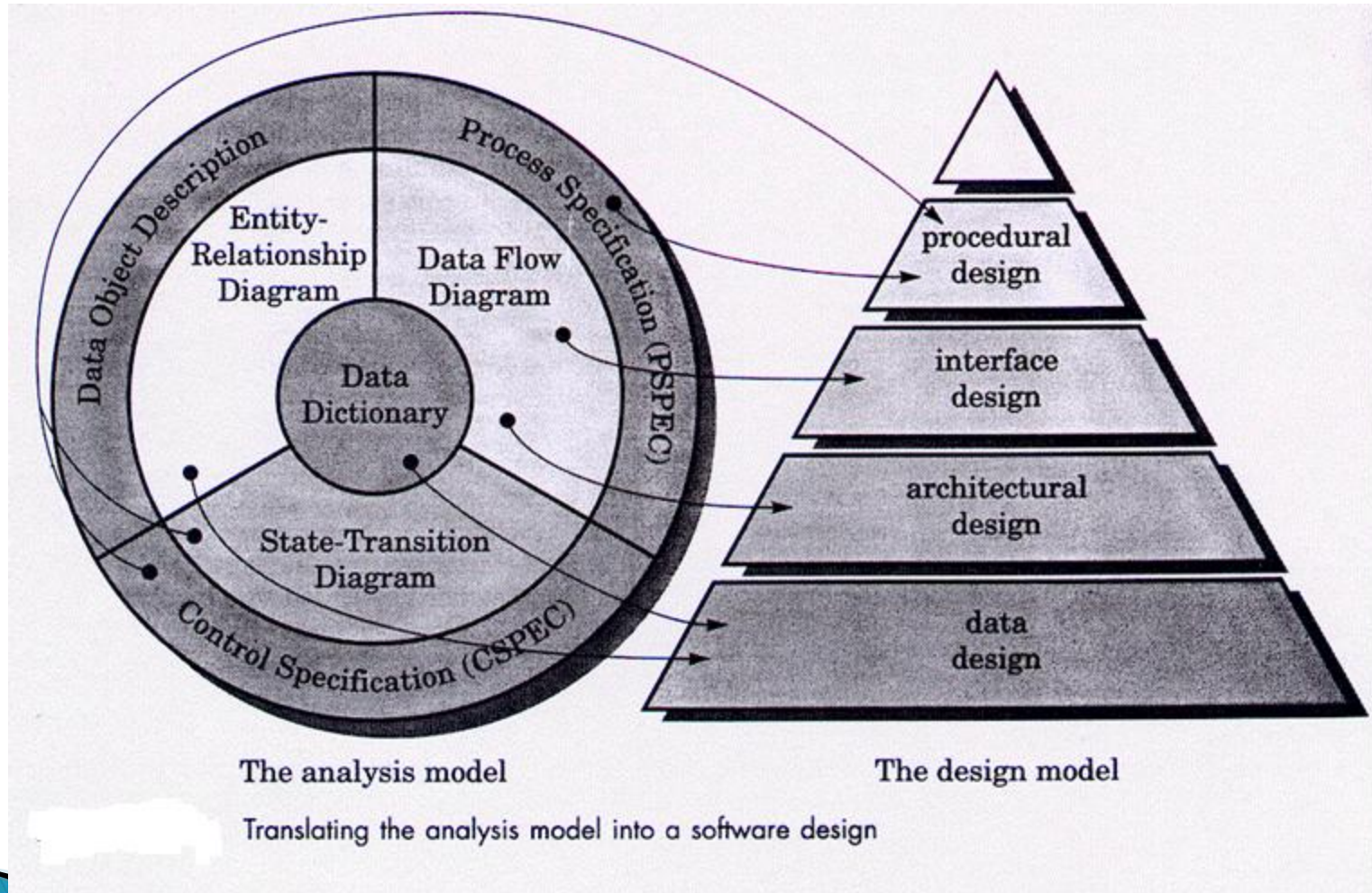
Software Design

- ▶ **Software Design -- An iterative process transforming requirements into a “blueprint” for constructing the software.**

Goals of the design process:

1. The design must implement all of the explicit requirements contained in the analysis model and it must accommodate all of the implicit requirements desired by the customer.
2. The design must be a readable, understandable guide for those who generate code and for those who test and subsequently support the software.
3. The design should address the data, functional and behavioral domains from an implementation perspective

Analysis to Design



Design model

Components of a design model :

▶ Data Design

- Transforms information domain model into data structures required to implement software

▶ Architectural Design

- Defines relationship among the major structural elements of a software

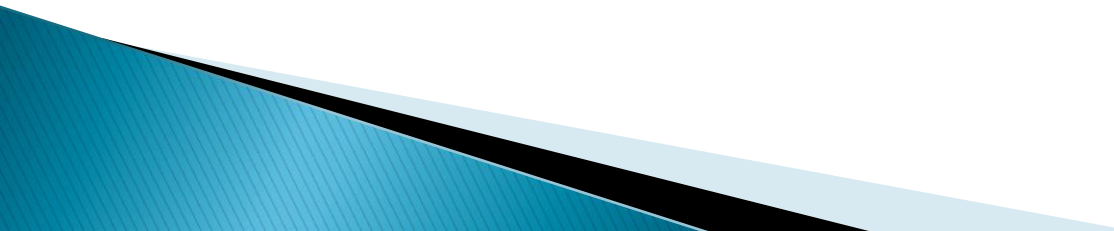
▶ Interface Design

- Describes how the software communicates with systems that interact with it and with humans.

▶ Procedural Design

- Transforms structural elements of the architecture into a procedural description of software components

Design Concepts(Abstraction)

- ▶ Wasserman: “Abstraction permits one to concentrate on a problem at some level of abstraction without regard to low level detail.
 - ▶ At the highest level of abstraction a solution is stated in broad terms using the language of the problem environment.
 - ▶ At lower level, a procedural orientation is taken.
 - ▶ At the lowest level of abstraction the solution is stated in a manner that can be directly implemented.
- 

Design Concepts(Abstraction)

Types of abstraction :

1. Procedural Abstraction :

A named sequence of instructions that has a specific & limited function

E.g: Word OPEN for a door

2. Data Abstraction :

A named collection of data that describes a data object.

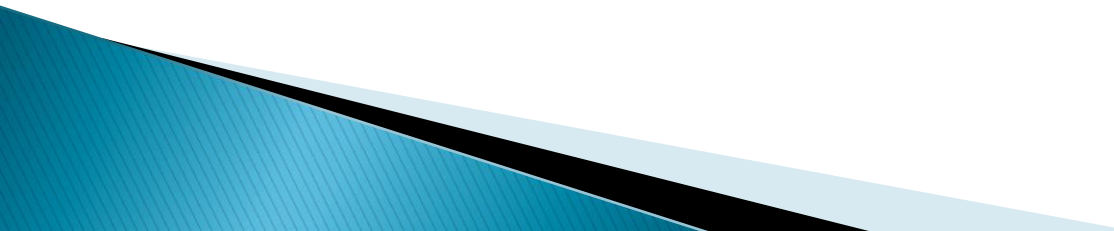
Data abstraction for door would be a set of attributes that describes the door

(e.g. door type, swing direction, weight, dimension)

3. Control Abstraction: implies control mechanism which hides the internal details.

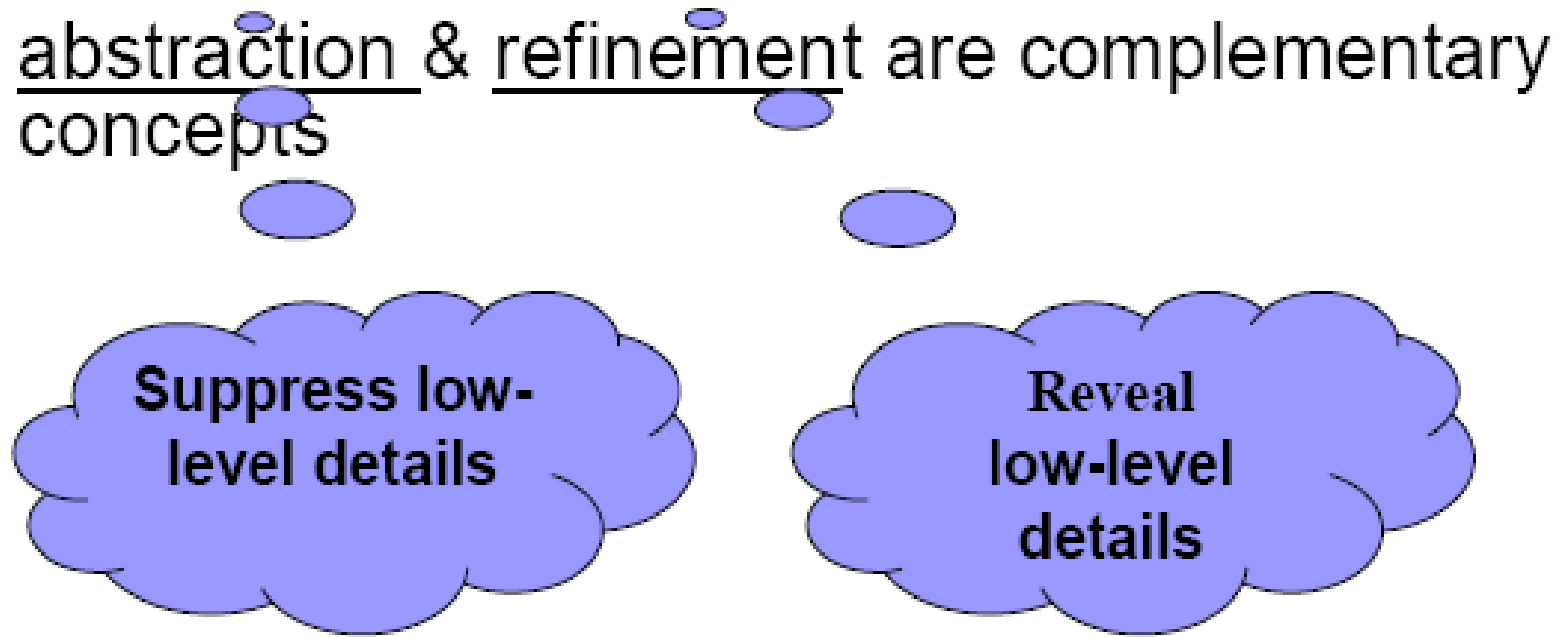
Design Concepts(Refinement)

2. Refinement

- Process of elaboration.
 - Top -down strategy.
 - Start with the statement of function defined at the abstract level, decompose the statement of function in a stepwise fashion until programming language statements are reached.
- 

Design Concepts(Refinement)

abstraction & refinement are complementary concepts



Suppress low-level details

Reveal low-level details

Design Concepts(Modularity)

- ▶ In this concept, software is divided into separately named and addressable components called modules
- ▶ Follows “divide and conquer” concept, a complex problem is broken down into several manageable pieces
- ▶ Let p_1 and p_2 be two problems.
- ▶ Let E_1 and E_2 be the effort required to solve them $--\rightarrow$
If $C(p_1) > C(p_2)$
Hence $E(p_1) > E(p_2)$

Design Concepts(Modularity)

Now--→

Complexity of a problem that combines p_1 and p_2 is greater than complexity when each problem is consider

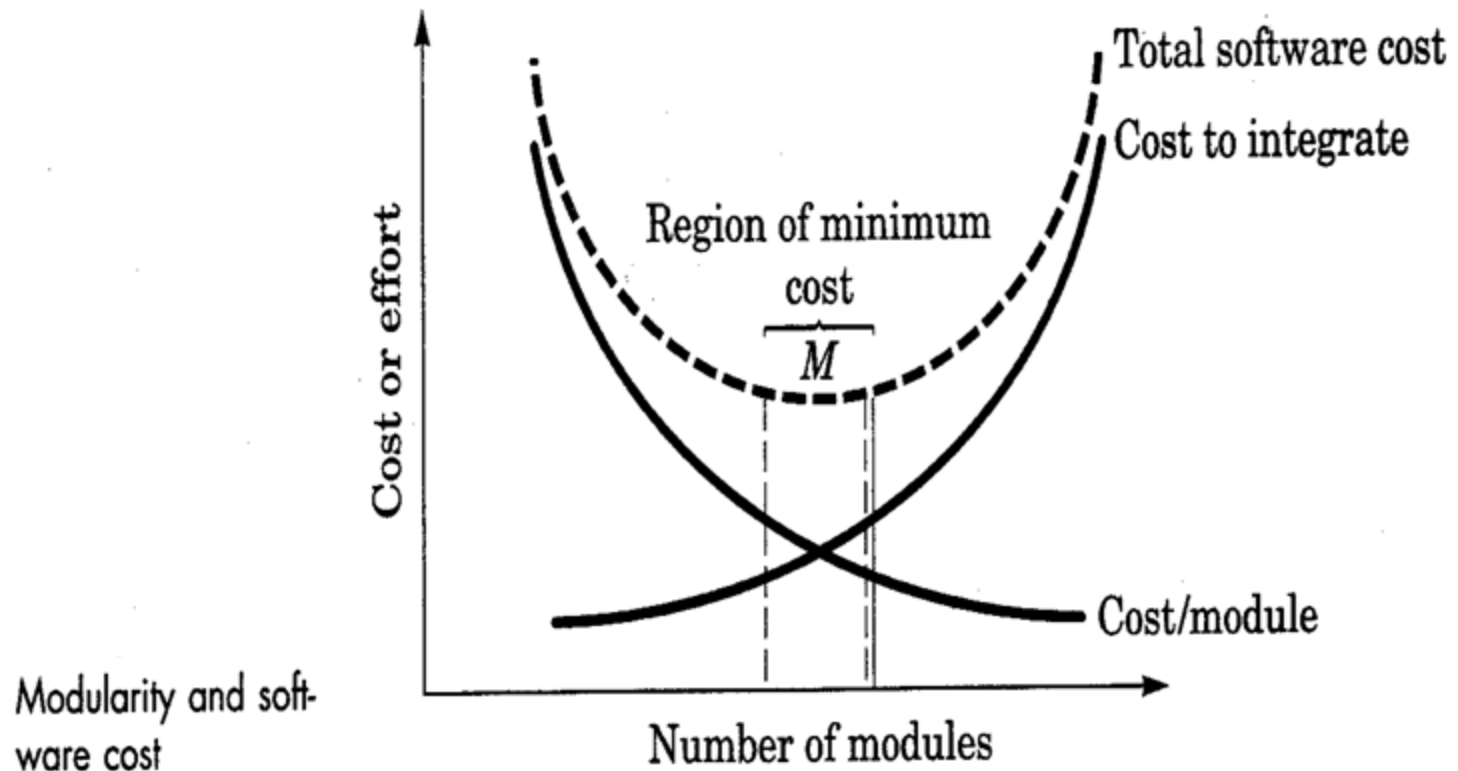
$$C(p_1 + p_2) > C(p_1) + C(p_2),$$

Hence

$$E(p_1 + p_2) > E(p_1) + E(p_2)$$

It is easier to solve a complex problem when you break it into manageable pieces

Design Concepts(Modularity)



Design Concepts(Modularity)

5 criteria to evaluate a design method with respect to its modularity-----→

Modular understandability

module should be understandable as a standalone unit
(no need to refer to other modules)

Modular continuity

If small changes to the system requirements result in changes to individual modules, rather than system wide changes, the impact of side effects will be minimized

Modular protection

- ▶ If an error occurs within a module then those errors are localized and not spread to other modules.

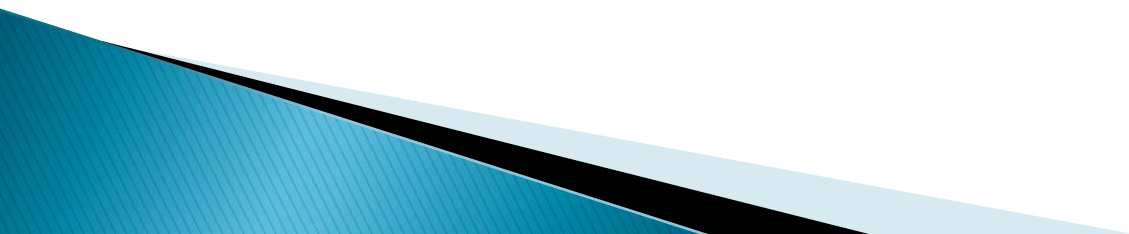
Design Concepts(Modularity)

Modular Composability

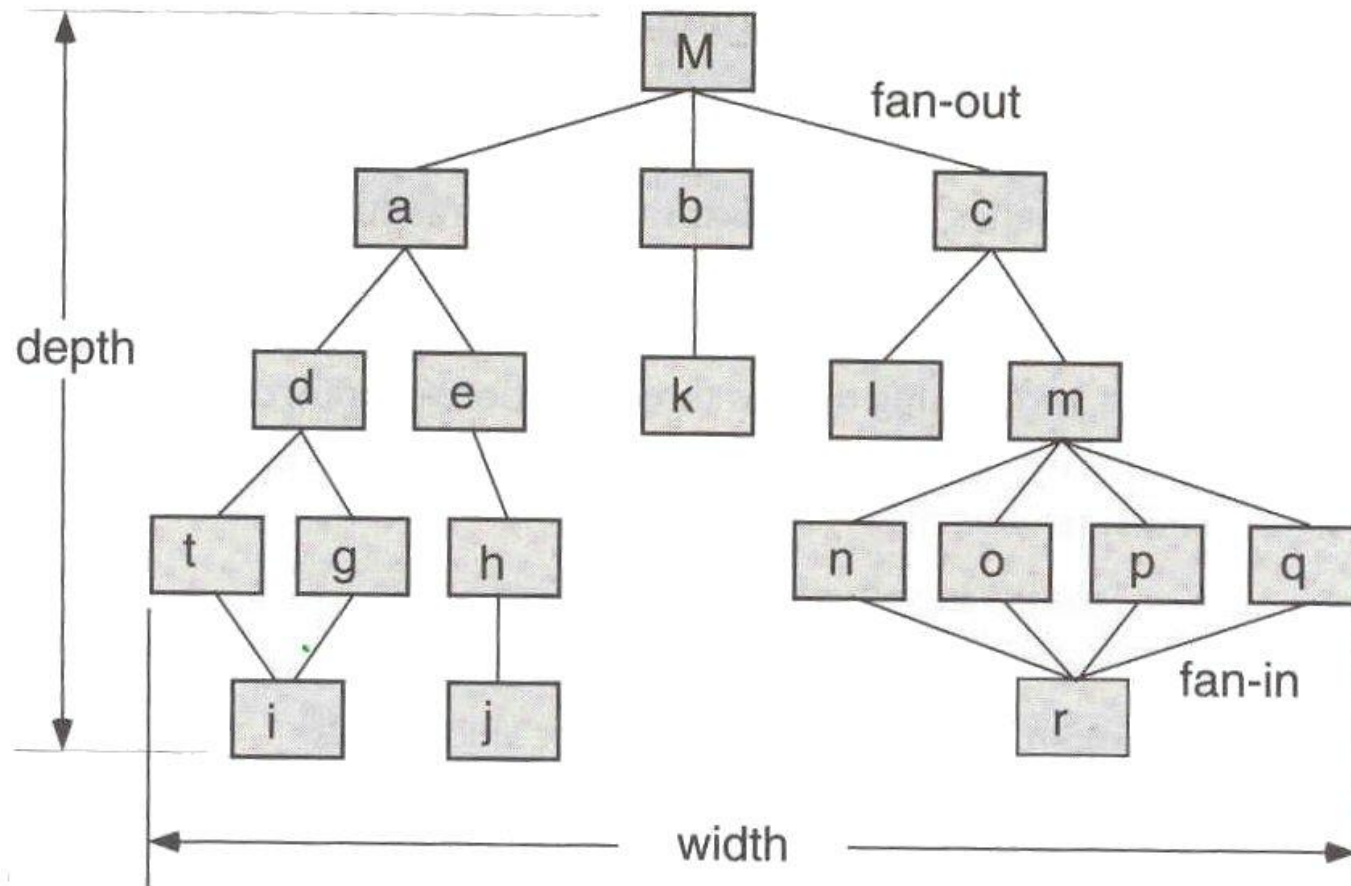
Design method should enable reuse of existing components.

Modular Decomposability

Complexity of the overall problem can be reduced if the design method provides a systematic mechanism to decompose a problem into sub problems



Design Concepts(Control Hierarchy)



Design Concepts(Control Hierarchy)

- ▶ Also called program structure
 - ▶ Represent the organization of program components.
 - ▶ Does not represent procedural aspects of software such as decisions, repetitions etc.
 - ▶ **Depth** –No. of levels of control (distance between the top and bottom modules in program control structure)
 - ▶ **Width**– Span of control.
 - ▶ **Fan-out** –no. of modules that are directly controlled by another module
 - ▶ **Fan-in** – how many modules directly control a given module
 - ▶ **Super ordinate** –module that control another module
 - ▶ **Subordinate** – module controlled by another
- 

Design Concepts(Control Hierarchy)

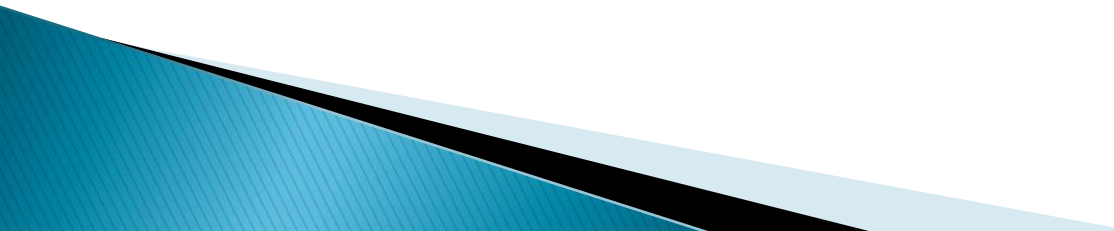
- Visibility –set of program components that may be called or used as data by a given component
- Connectivity – A module that directly causes another module to begin execution is connected to it.

Design Concepts (Data Structure)

Data structure is a representation of the logical relationship among individual elements of data.

- **Scalar item –**

A single element of information that may be addressed by an identifier .

- **Scalar item organized as a list– array**
 - **Data structure that organizes non–contiguous scalar items–linked list**
- 

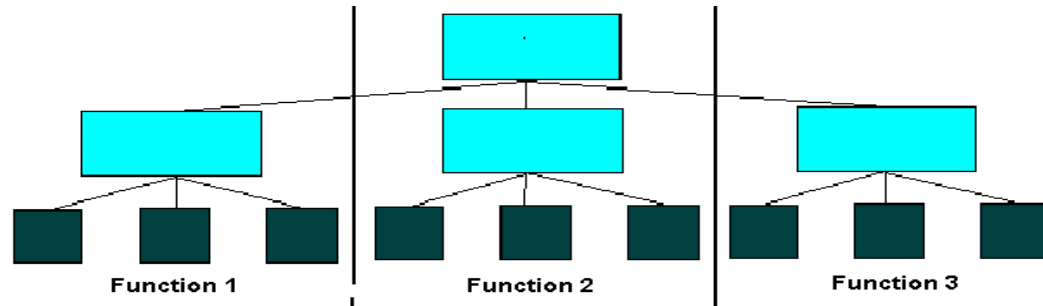
Design Concepts (Structural partitioning)

If the architectural style of a system is hierarchical, program structure can be partitioned -----→

1. Horizontal partitioning
2. Vertical partitioning

Design Concepts (Structural partitioning)

Horizontal partitioning



Separate branches can be defined for each major function

Eg : 3 partitions

1. Input
2. Data Transformation
3. Output

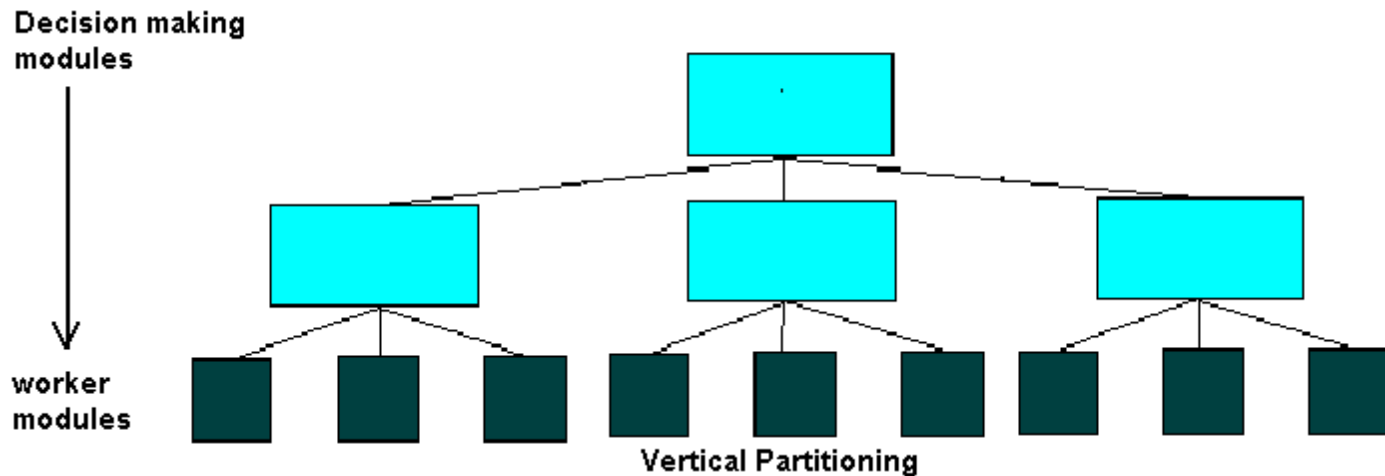
Advantages

- Easier to test
- Easier to maintain
- Propagation of fewer side effects
- Easier to add new features

Design Concepts (Structural partitioning)

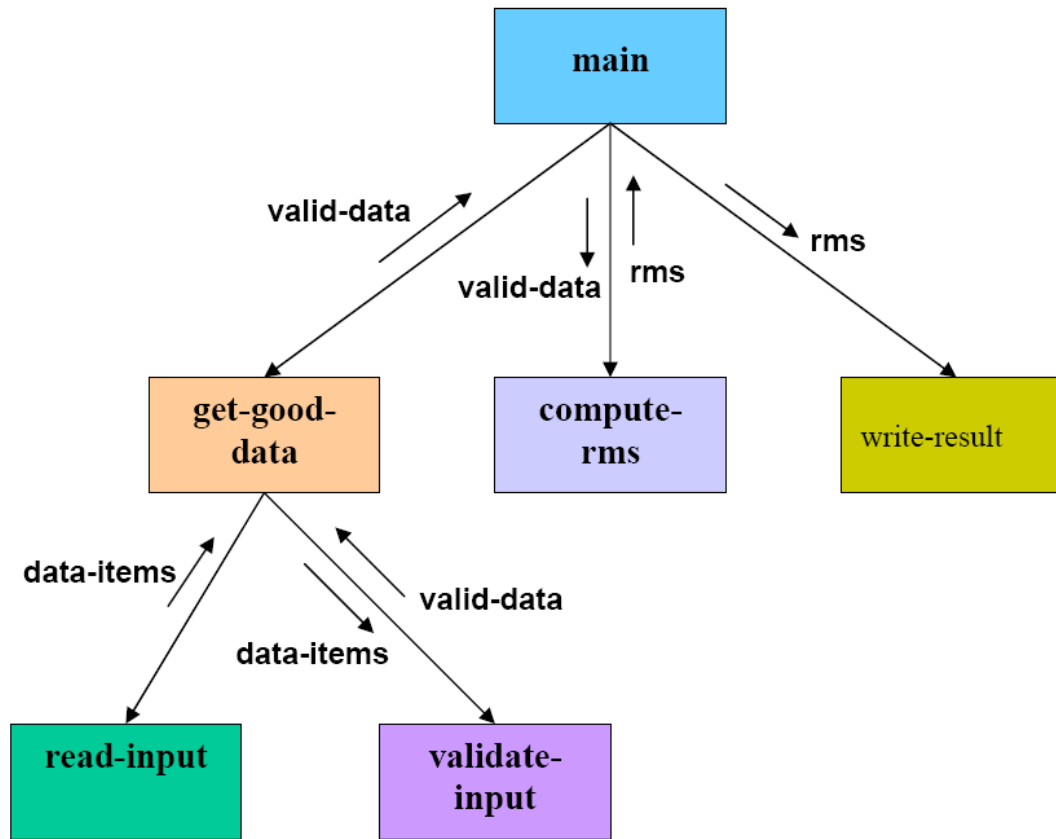
Vertical Partitioning

- Control and work modules are distributed top down
- Top level modules perform control functions
- Lower modules perform computations (input processing and output)



Design Concepts (Structural partitioning)

Eg of Horizontal Partitioning



Design Concepts (Structural partitioning)

Eg of Vertical Partitioning

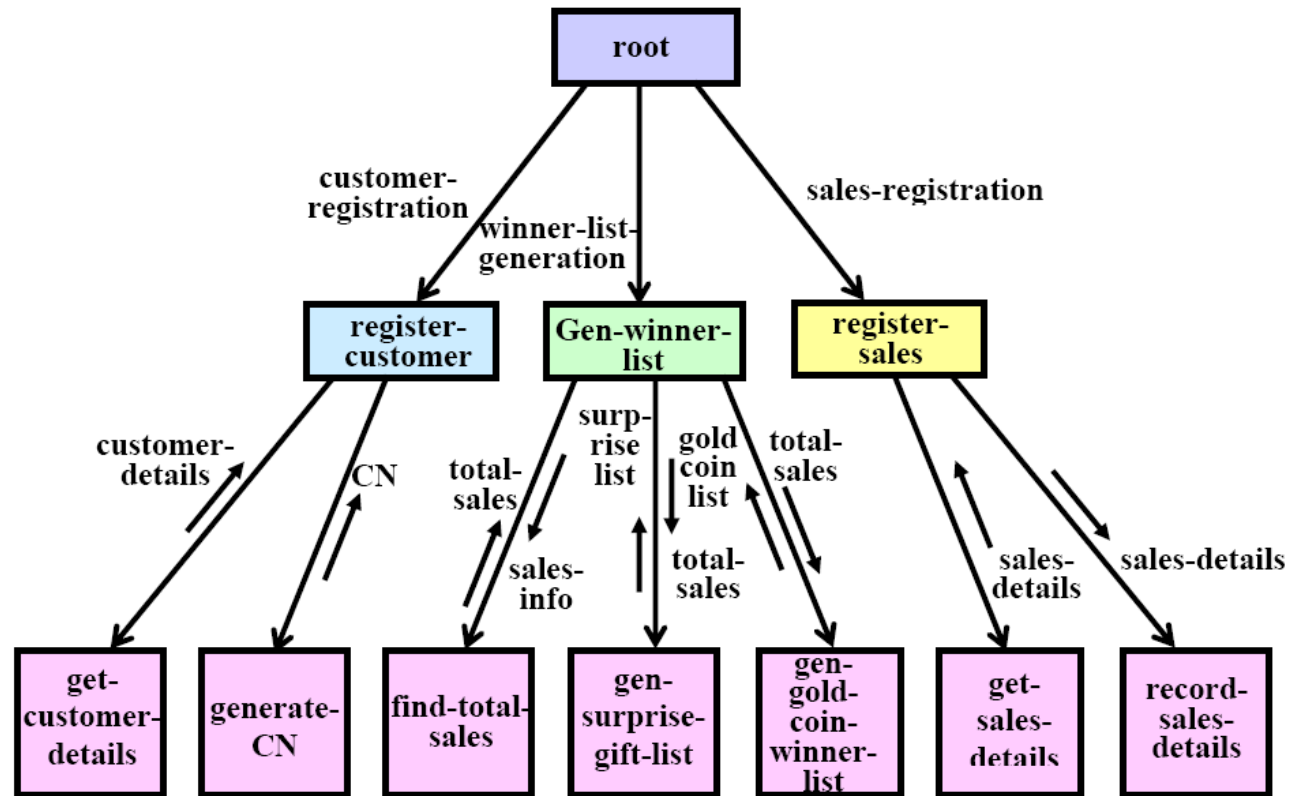
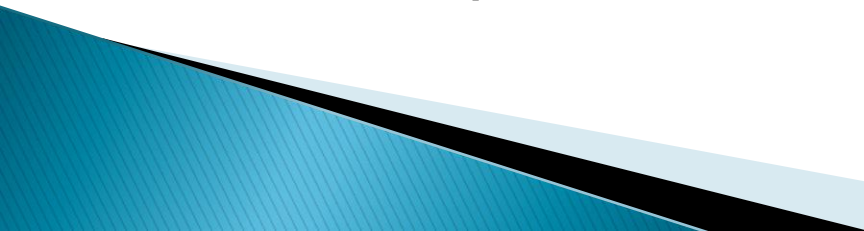


Fig. 36.13 Structure chart for the supermarket prize scheme

Functional Independence

- ▶ Functional independence is achieved by developing modules with “single minded” function and an aversion to excessive interaction with other modules.
 - ▶ Measured using 2 qualitative criteria:
 1. Cohesion : Measure of the relative strength of a module.
 2. Coupling : Measure of the relative interdependence among modules.
- 

Cohesion

- Strength of relation of elements within a module
- Element– Group of instructions or a data definition.
- Strive for high cohesion

Different types of cohesion :

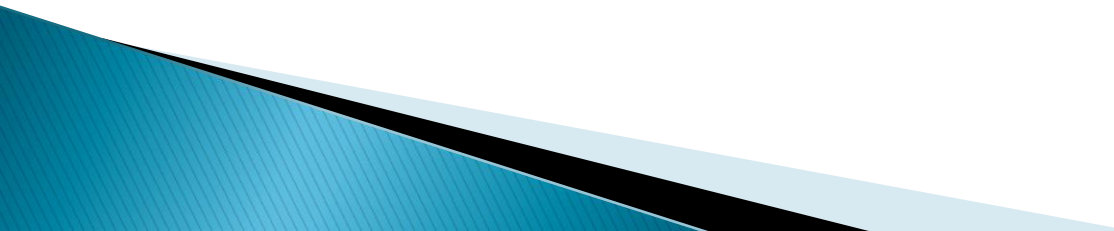
1. Functional Cohesion (Highest):

A functionally cohesive module contains elements that all contribute to the execution of one and only one problem-related task.

Examples of functionally cohesive modules are

Compute cosine of an angle

Calculate net employee salary



Cohesion

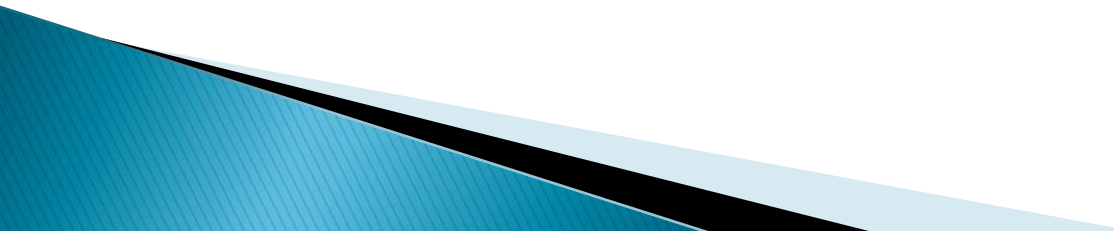
2. Sequential Cohesion :

A *sequentially cohesive* module is one whose elements are involved in activities such that output data from one activity serves as input data to the next.

Eg: Module read and validate customer record

- ▶ Read record
- ▶ Validate customer record

Here output of one activity is input to the second



Cohesion

3. Communicational cohesion

A *communicationally cohesive* module is one whose elements contribute to activities that use the same input or output data. Suppose we wish to find out some facts about a book

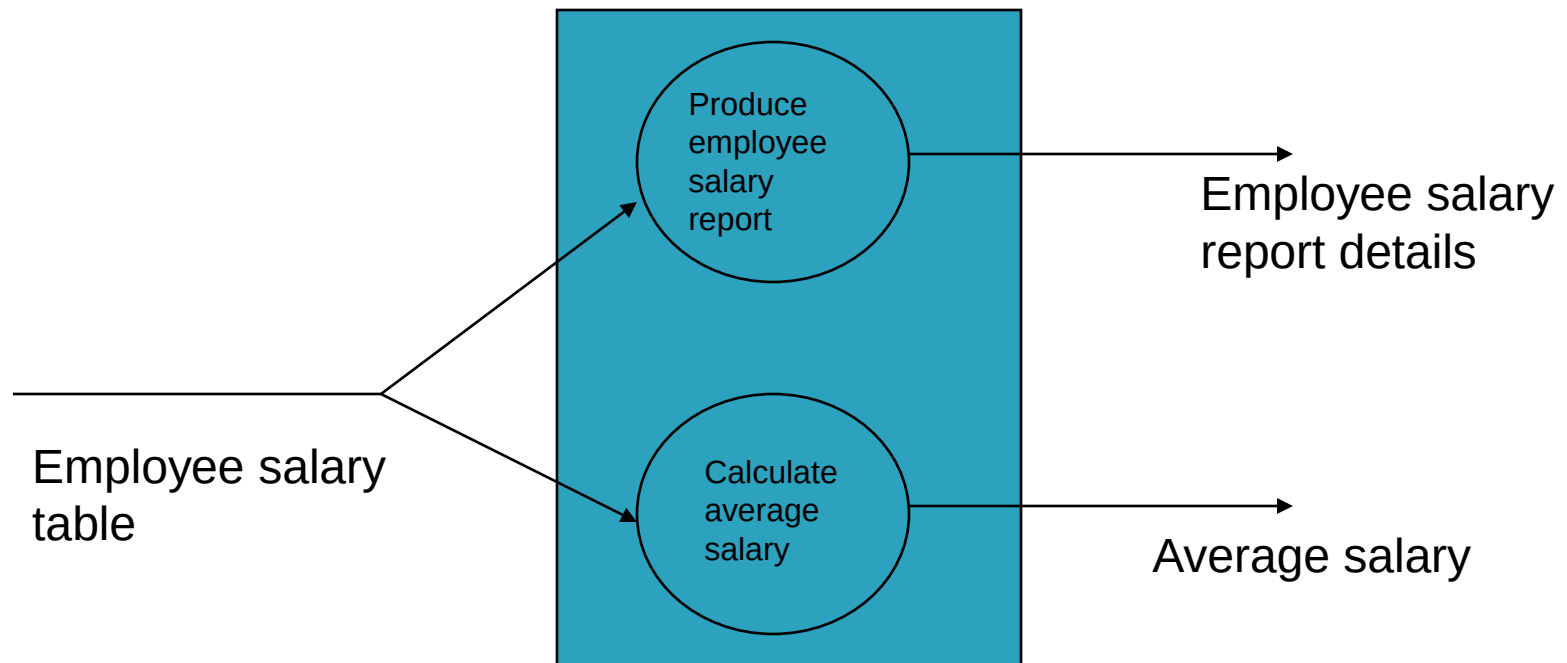
For instance, we may wish to

- ▶ FIND TITLE OF BOOK
- ▶ FIND PRICE OF BOOK
- ▶ FIND PUBLISHER OF BOOK
- ▶ FIND AUTHOR OF BOOK

These four activities are related because they all work on the same input data, the book, which makes the “module” communicationally cohesive.

Cohesion

Eg: module which produces employee salary report and calculates average salary



Cohesion

4. Procedural Cohesion

A procedurally cohesive module is one whose elements are involved in different and possibly unrelated activities in which control flows from each activity to the next.

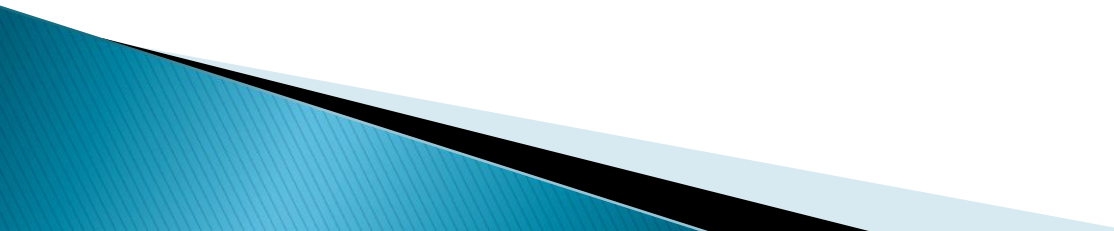
A piece of coding ties the activities together

Eg:

Module that averages two completely unrelated tables, **TABLE-A** and **TABLE-B**, which both just happen to have 100 elements each.

Cohesion

```
module compute table-A-avg and table-B-avg
uses table-A, table-B
returns table-A-avg, table-B-avg
  table-A-total : = 0
  table-B-total : = 0
  for i = 1 to 100
    add table-A (i) to table-A-total
    add table-B (i) to table-B-total
  endfor
  table-A-avg : = table-A-total/100
  table-B-avg : = table-B-total/100
endmodule
```



Cohesion

5. Temporal Cohesion

A temporally cohesive module is one whose elements are involved in activities that are related in time.

A module **INITIALIZE** initializes many different functions in a mighty sweep, causing it to be broadly related to several other modules in the system.

```
module initialize
    updates a-counter, b-counter, items table,
           totals table, switch-a, switch-b
    rewind tape-a
    set a-counter to 0
    rewind tape-b
    set b-counter to 0
    clear items table
    clear totals table
    set switch-a to off
    set switch-b to on
endmodule
```

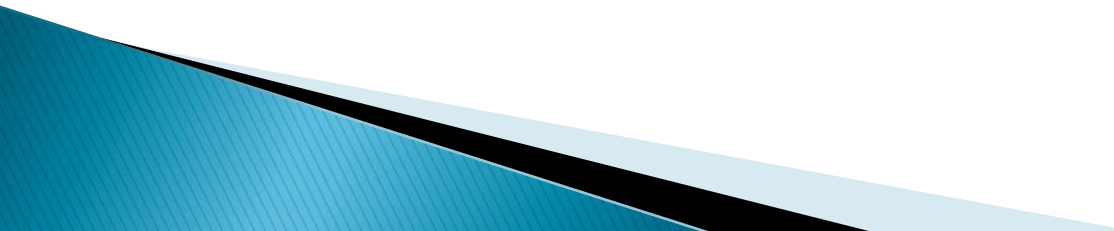
Cohesion

6. Logical Cohesion

A logically cohesive module is one whose elements contribute to activities of the same general category in which the activity or activities to be executed are selected from outside the module.

A logically cohesive module contains a number of activities of the same general kind.

To use the module, we pick out just the piece(s) we need.



Cohesion

Eg. A module maintain employee master file

Uses input flag/* to choose which function*/

If input flag =1

{Add a new record to master file}

If input flag =2

{delete an employee record }

If input flag=3

{edit employee record }

–

–

Endmodule

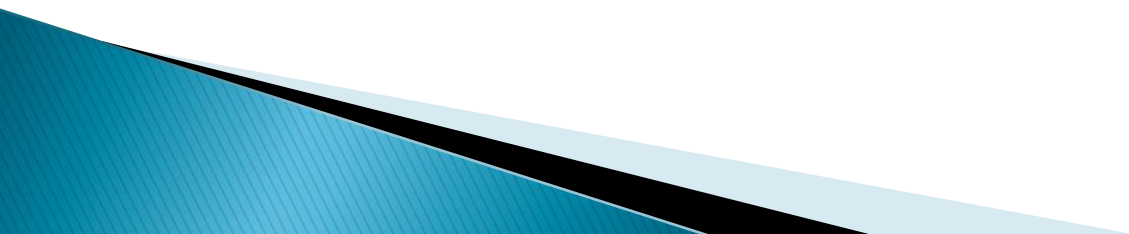


Cohesion

7. Coincidental Cohesion (Lowest)

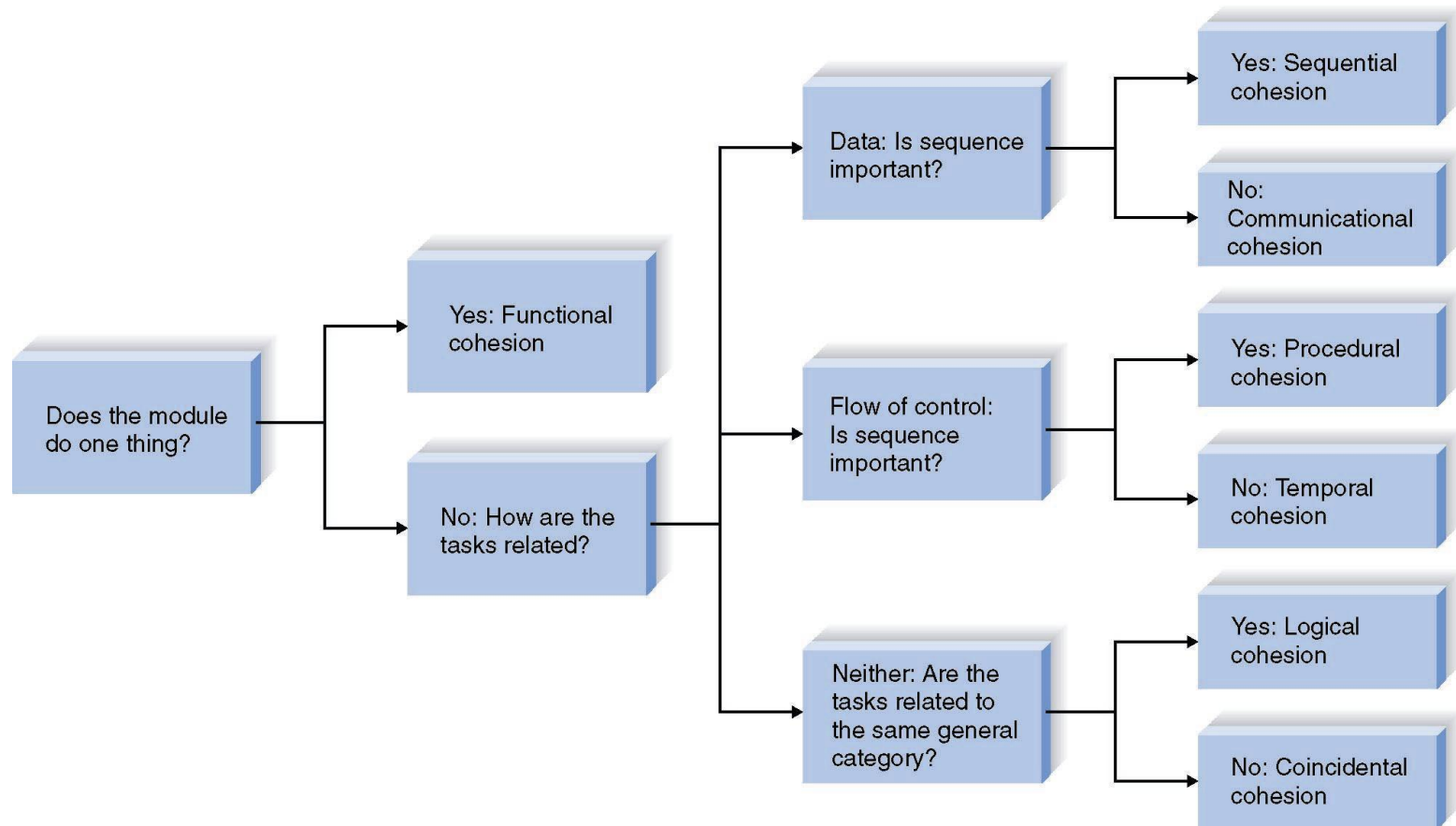
A coincidentally cohesive module is one whose elements contribute to activities with no meaningful relationship to one another.

Eg. When a large program is modularized by arbitrarily segmenting the program into several small modules.



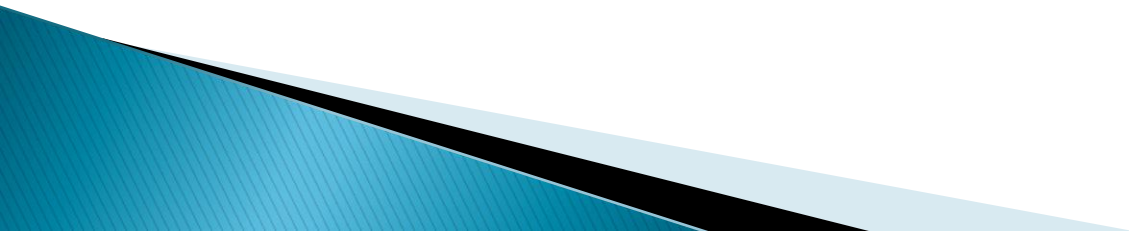
Cohesion

A decision tree to show module cohesion



Coupling

- ▶ Measure of interconnection among modules in a software structure
- ▶ Strive for lowest coupling possible.



Coupling

Types of coupling

1. Data coupling (Most desirable)

Two modules are data coupled, if they communicate through parameters where each parameter is an elementary piece of data.

e.g. an integer, a float, a character, etc. This data item should be problem related and not be used for control purpose.

2. Stamp Coupling

Two modules are said to be stamp coupled if a data structure is passed as parameter but the called module operates on some but not all of the individual components of the data structure.

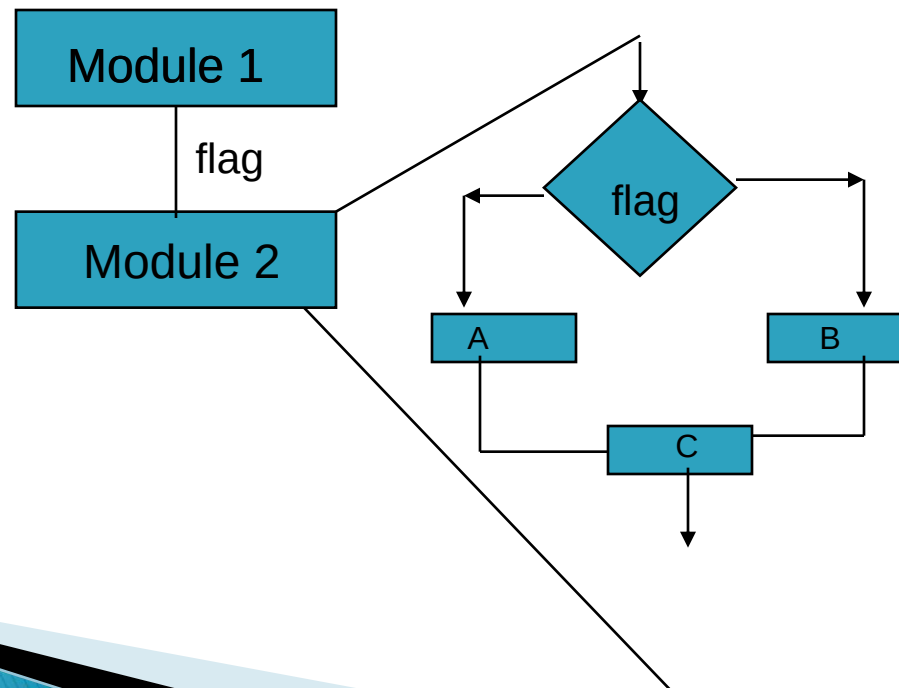
Coupling

3. Control Coupling

Two modules are said to be control coupled if one module passes a control element to the other module.

This control element affects / controls the internal logic of the called module

Eg: flags



Coupling

4. Common Coupling

Takes place when a number of modules access a data item in a global data area.

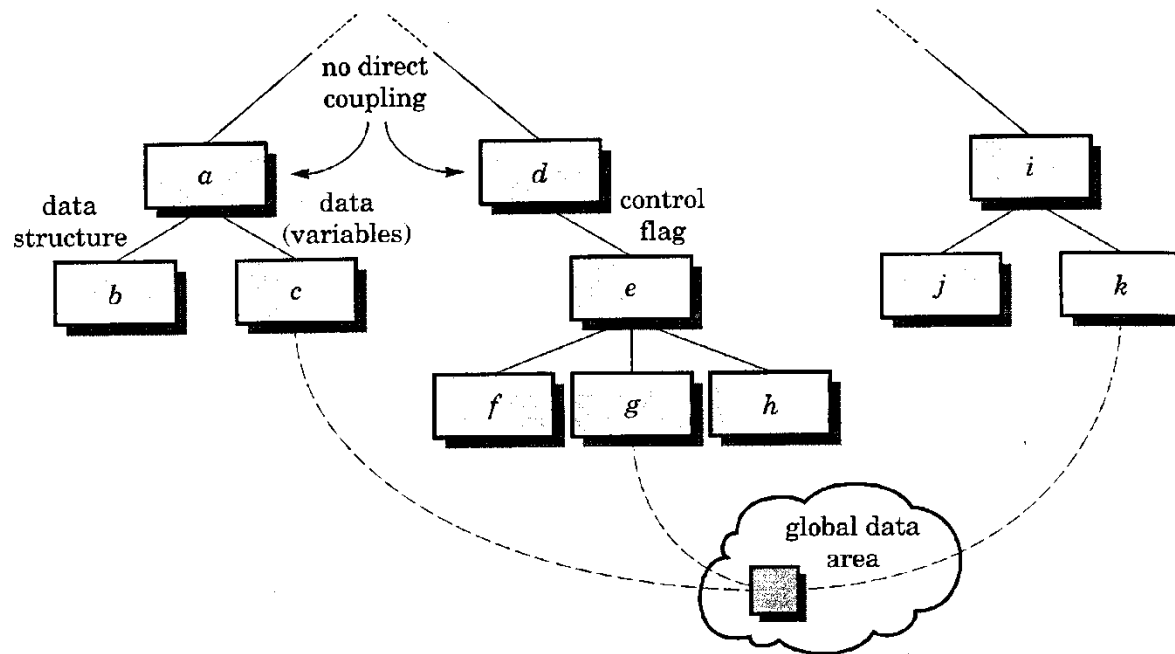


FIGURE 13.8. Types of coupling

Modules *c*, *g* and *k* exhibit common coupling

Coupling

5. Content coupling (Least desirable)

Two modules are said to be content coupled if one module branches into another module or modifies data within another.

Eg:

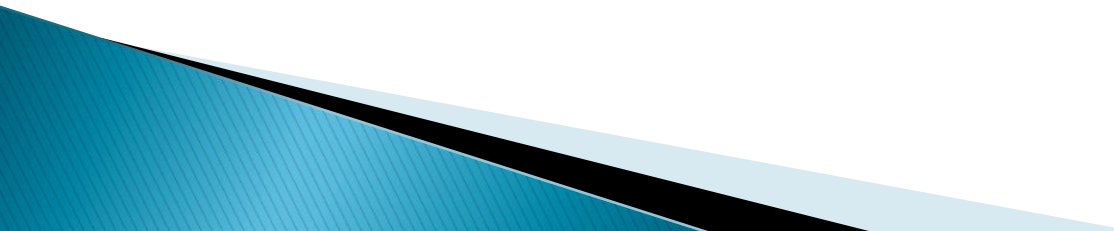
```
int func1(int a)
{ printf("func1");
  a+=2;
  goto F2A;
return a;
}
```

```
void func2(void)
{ printf("func2");
  F2A : printf("At F2A")
}
```

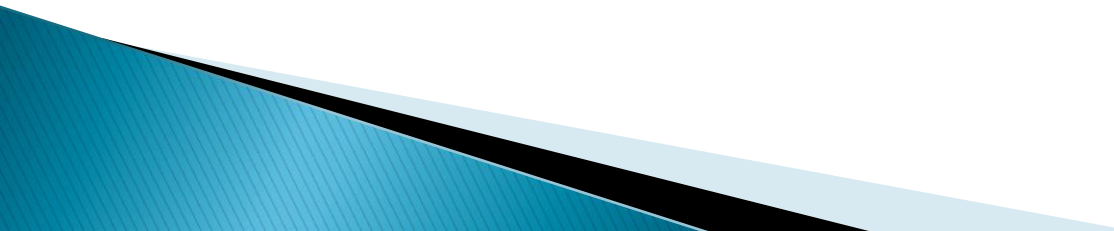
Design principles:

- ▶ The design process should not suffer from ‘tunnel vision’
A good designer should consider alternative approaches.
 - ▶ The design should be traceable to the analysis model
 - ▶ The design should not reinvent the wheel
Time is short and resources are limited! Hence use well tested and reusable s/w components
 - ▶ The design should ‘minimize the intellectual distance” between the S/W and the problem as it exists in the real world
That is, the structure of the software design should (whenever possible) mimic the structure of the problem domain.
 - ▶ The design should exhibit uniformity & integration
A design is uniform if it appears that one person developed the whole thing.
- 

Design principles:

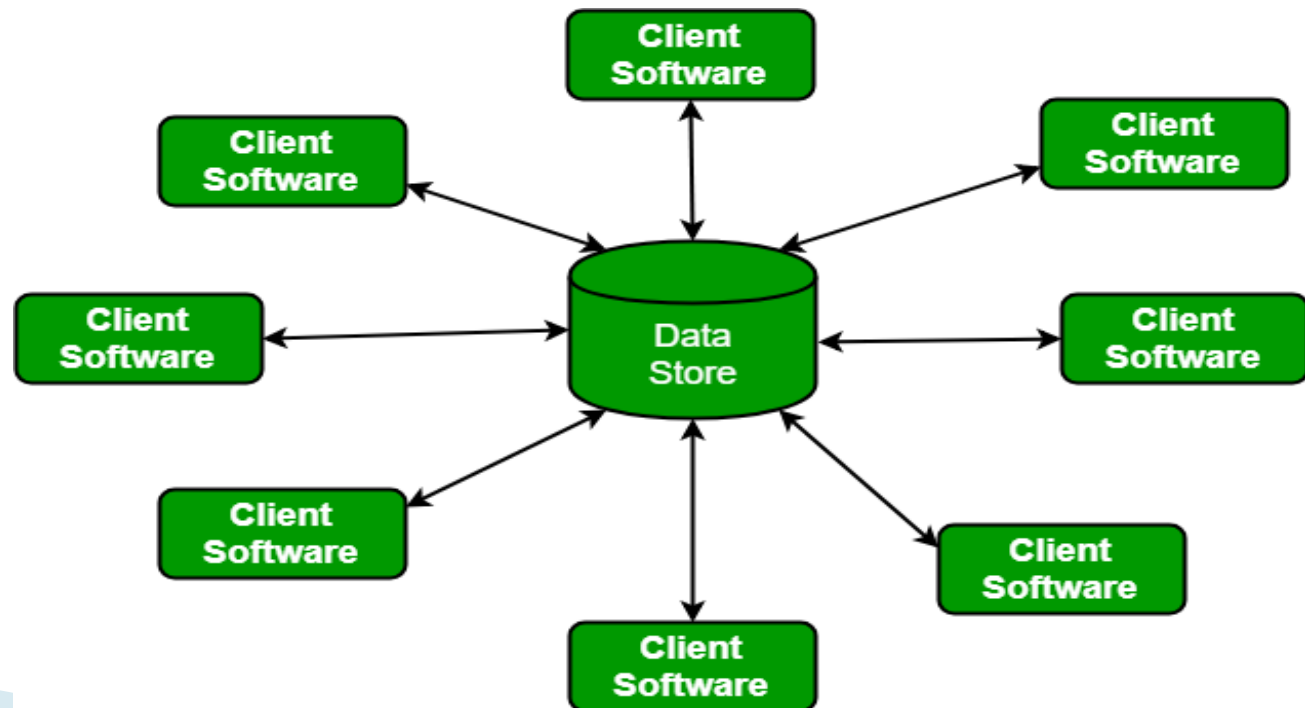
- ▶ The design should be structured to accommodate change
Upcoming modifications and improvements should not lead to drastic changes or redesign of software
 - ▶ The design should be structured to degrade gently, even when aberrant data, events, or operating conditions are encountered
Use message/progress bars whenever possible
 - ▶ Design is not coding, coding is not design
 - ▶ The design should be assessed for quality as it is being created, not after the fact .
 - ▶ The design should be reviewed to minimize conceptual (semantic) errors
- 

Architectural Styles

- ▶ IEEE defines architectural design as “the process of defining a collection of hardware and software components and their interfaces to establish the framework for the development of a computer system.”
 - ▶ The software that is built for computer-based systems can exhibit one of these many architectural styles:
 - Data Centered Architecture
 - Data flow Architecture
 - Call and Return Architecture
 - Object Oriented Architecture
 - Layered Architecture
- 

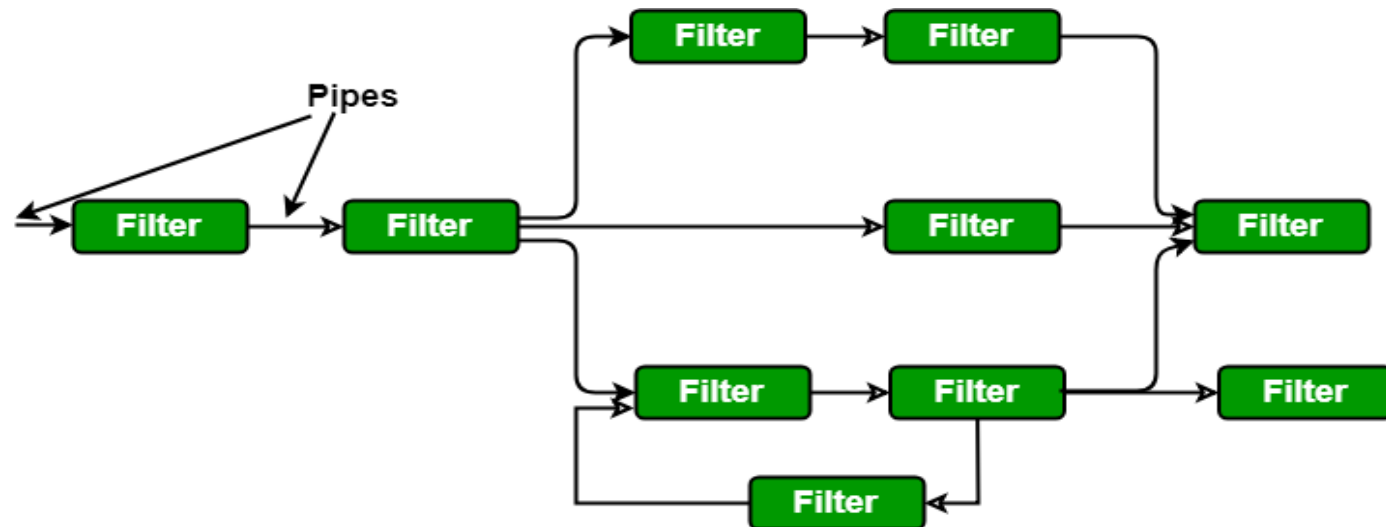
Data Centered Architecture

- ▶ A data store will reside at the center of this architecture and is accessed frequently by the other components that update, add, delete or modify the data present within the store.



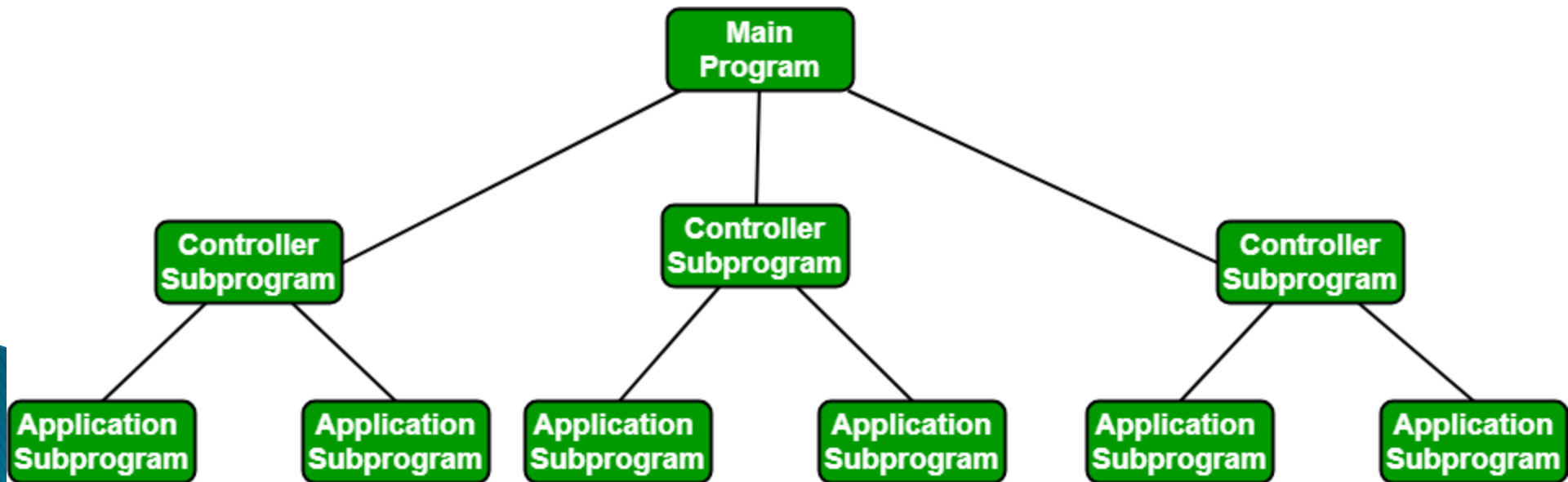
Data flow architecture

- ▶ This kind of architecture is used when input data to be transformed into output data through a series of computational manipulative components.
- ▶ The figure represents pipe-and-filter architecture since it uses both pipe and filter and it has a set of components called filters connected by pipes.



Call and return architecture

- ▶ It is used to create a program that is easy to scale and modify.
- ▶ 2 types:
 - Remote procedure call architecture
 - Main program or Subprogram architectures

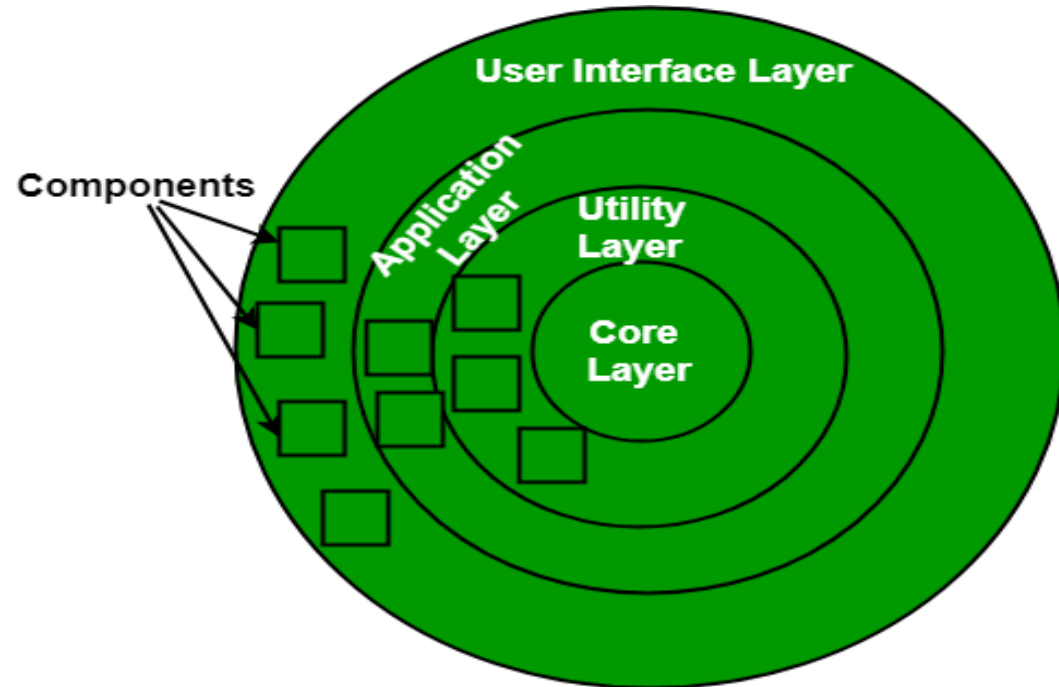


Object-oriented architecture

- ▶ The components of a system encapsulate data and the operations that must be applied to manipulate the data.
- ▶ The coordination and communication between the components are established via the message passing.

Layered architecture

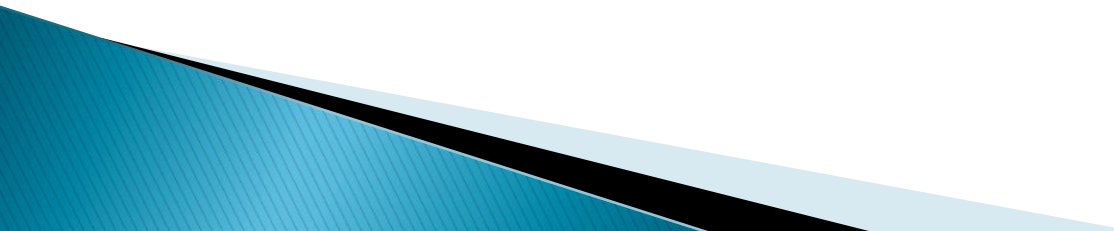
- ▶ A number of different layers are defined with each layer performing a well-defined set of operations.
- ▶ Each layer will do some operations that becomes closer to machine instruction set progressively.



User Interface Design

- ▶ Interface design focuses on three areas of concern:
 - the design of interfaces between software components,
 - the design of interfaces between the software and other nonhuman producers and consumers of information (i.e., other external entities),
 - And the design of the interface between a human (i.e., the user) and the computer.
- ▶ This topic focuses on last category: User Interface Design

User Interface Design

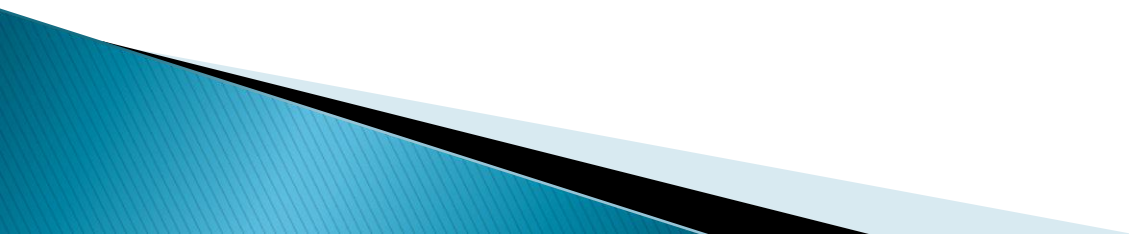
- ▶ User interface design creates an effective communication medium between a human and a computer.
 - ▶ Following a set of interface design principles, design identifies interface objects and actions and then creates a screen layout that forms the basis for a user interface prototype.
- 

3 Golden Rules to UI Design

Theo Mandel suggested following rules:

- ▶ Place the user in control.
- ▶ Reduce the user's memory load.
- ▶ Make the interface consistent.

These golden rules actually form the basis for a set of user interface design principles.



Place the user in control

- ▶ Let the user control the system and not vice-versa.
- ▶ Create an interface that is easy to build but also easy to use – trade off between the two.

Place the user in control –design principles

- ▶ **Define interaction modes in a way that does not force a user into unnecessary or undesired actions.**
 - Interaction mode-current state of interface.
 - E.g. spell check
 - Don't force user to remain in current mode.
- ▶ **Provide for flexible interaction.**
 - Different users have different interaction preferences.
 - E.g. Interact through various ways.

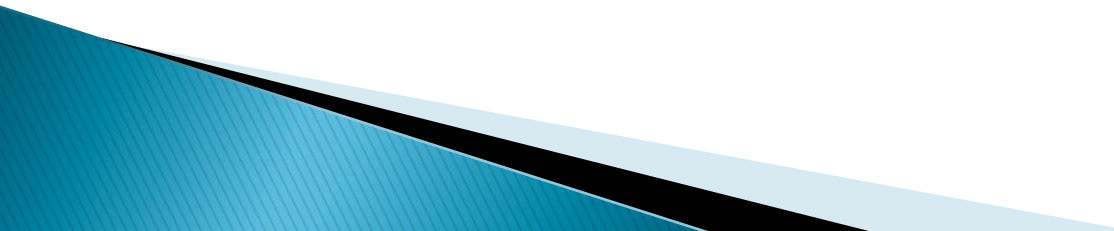
Place the user in control –design principles

- ▶ **Allow user interaction to be interruptible and undoable.**
 - user should be able to interrupt the sequence and undo the things.
- ▶ **Streamline interaction as skill levels advance and allow the interaction to be customized.**
 - Generalize same sequence of repeated instructions.
 - E.g. macros
- ▶ **Hide technical internals from the casual user.**
 - Show virtual world to user.
 - Hide technical details.

Place the user in control –design principles

- ▶ **Design for direct interaction with objects that appear on the screen.**
 - Let user feel sense of control while manipulating objects as if object were a physical thing.
 - E.g. stretching an object (scaling by size)

Reduce the user's memory load

- ▶ The more a user has to remember, the more error-prone will be the interaction with the system.
 - ▶ Whenever possible, the system should “remember” pertinent information and assist the user with an interaction scenario that assists recall.
- 

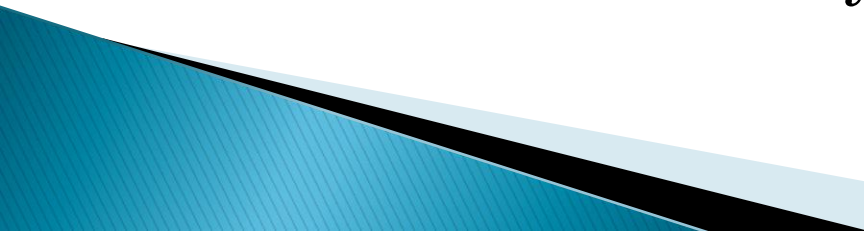
Reduce the user's memory load- design principles

- ▶ **Reduce demand on short-term memory**
 - reduce the requirements to remember past actions and results.
 - Provide visual cues to recognize rather than to recall.
- ▶ **Establish meaningful defaults**
 - user should be allowed to change initial set of defaults.
 - Even reset option should be given.
- ▶ **Define shortcuts that are intuitive**
 - Bind mnemonics to the function in a way to remember
 - E.g. alt-P to print

Reduce the user's memory load- design principles

- ▶ **The visual layout of the interface should be based on a real world metaphor**
 - enable the user to rely on well understood visual cues to remember interaction sequences.
 - E.g. bill payment process
- ▶ **Disclose information in a progressive fashion.**
 - Organize interface hierarchically
 - Information about task, object, function—present a higher abstraction level.
 - Show more detail if user is interested.
 - E.g. underlining function in word processor.

Make the interface consistent

- ▶ The interface should present and acquire information in a consistent fashion.
 - ▶ All visual information is organized according to a design standard that is maintained throughout all screen displays,
 - ▶ input mechanisms are constrained to a limited set that are used consistently throughout the application,
 - ▶ And mechanisms for navigating from task to task are consistently defined and implemented.
- 

Make the interface consistent – design principles

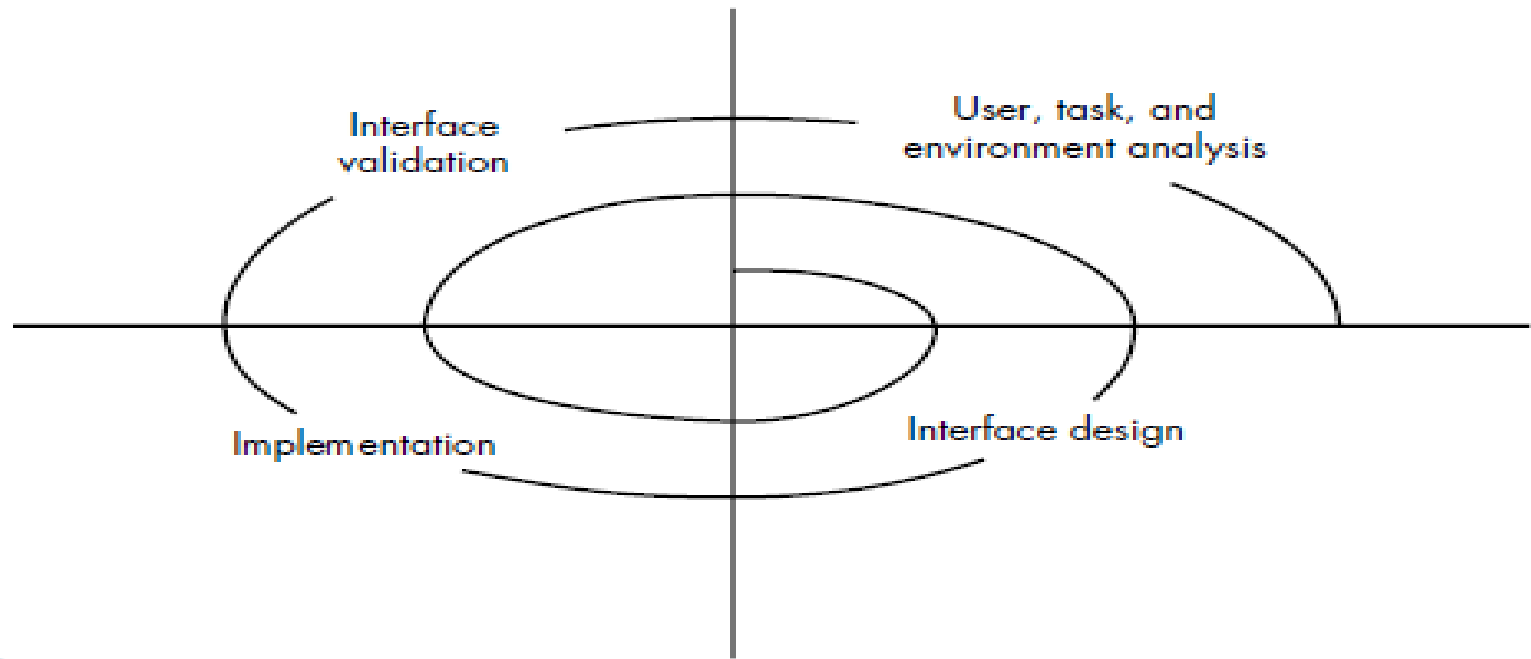
- ▶ **Allow the user to put the current task into a meaningful context**
 - Interfaces implement complex layers of interactions
 - Provide indicators
 - Allow navigation
- ▶ **Maintain consistency across a family of applications**
 - Set of applications should implement same design rules to maintain consistency.

Make the interface consistent – design principles

- ▶ If past interactive models have created user expectations, do not make changes unless there is a compelling reason to do so
 - Don't change a standard interactive sequence.
 - E.g. alt-S

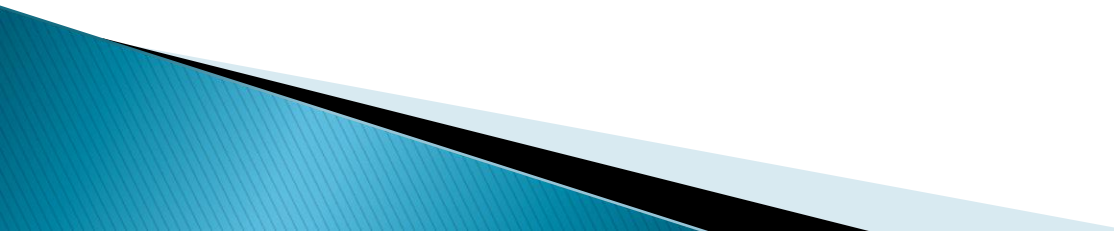
The user interface design process

- ▶ The design process for user interfaces is iterative and can be represented using a spiral model as shown



The user interface design process

Four distinct framework activities:

- ▶ **User, task, and environment analysis and modeling**
 - ▶ **Interface design**
 - ▶ **Interface construction**
 - ▶ **Interface validation**
- 

User, task, and environment analysis and modeling

- ▶ The initial analysis activity focuses on the profile of the users who will interact with the system.
 - ▶ Skill level, business understanding, and general receptiveness to the new system are recorded;
 - ▶ Different user categories are defined.
 - ▶ For each user category, requirements are elicited.
 - ▶ i.e. for each class of user, system perception is understood.
- 

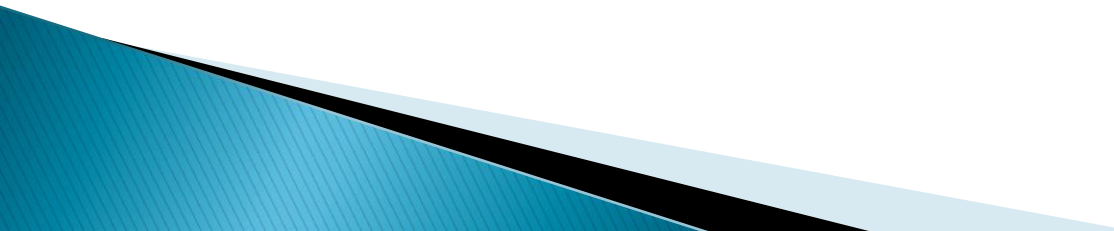
User, task, and environment analysis and modeling

- ▶ Those tasks that the user performs to accomplish the goals of the system are identified, described, and elaborated (over a number of iterative passes through the spiral).

User, task, and environment analysis and modeling

- ▶ The analysis of the user environment focuses on the physical work environment.
 - ▶ Ask following questions:
 - Where will the interface be located physically?
 - Will the user be sitting, standing, or performing other tasks unrelated to the interface?
 - Does the interface hardware accommodate space, light, or noise constraints?
 - ▶ The information gathered as part of the analysis activity is used to create an analysis model for the interface.
- 

Interface design

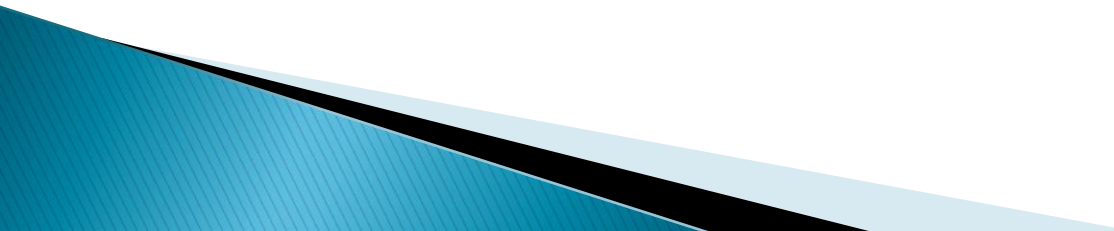
- ▶ The goal of interface design is to define a set of interface objects and actions (and their screen representations).
 - ▶ It enable a user to perform all defined tasks in a manner that meets every usability goal defined for the system.
- 

Interface construction

- ▶ The implementation activity begins with the creation of a prototype
- ▶ Usage scenarios are to be evaluated.
- ▶ As the iterative design process continues, a user interface tool kit (CASE tool for UI) may be used to complete the construction of the interface.

Interface validation

Validation focuses on

- ▶ the ability of the interface to implement every user task correctly, to accommodate all task variations, and to achieve all general user requirements;
 - ▶ the degree to which the interface is easy to use and easy to learn; and
 - ▶ the users' acceptance of the interface as a useful tool in their work.
- 

User Interface Standards

- ▶ **ISO 9241** is a multi-part standard from the International Organization for Standardization (ISO) covering ergonomics of human-computer interaction.
- ▶ It is managed by the ISO Technical Committee 159.
- ▶ It was originally titled *Ergonomic requirements for office work with visual display terminals (VDTs)*.